

表达式规则：

$$\text{cellx}[0, L_k:l_k, L_{k-1}:l_{k-1}, \dots, L_1:l_k; `0, T_k:t_k, T_{k-1}:t_{k-1}, \dots, T_1:t_1]$$

从上面的层次坐标可以看出，完整的层次坐标是从根上开始的，而0 则代表根坐标。

&运算符

描述：

从前面的介绍可知，单元格是可以扩展的，一个格子可以扩展出多个，那么如何知道某个扩展出来的格子在所有扩展出来的格子中排第几呢？此时我们就引入了&运算符，他可以获得当前格所属的某主格在所有扩展出来的格子中排第几，这种运算我们也称为层次坐标的逆运算。

表达式规则：

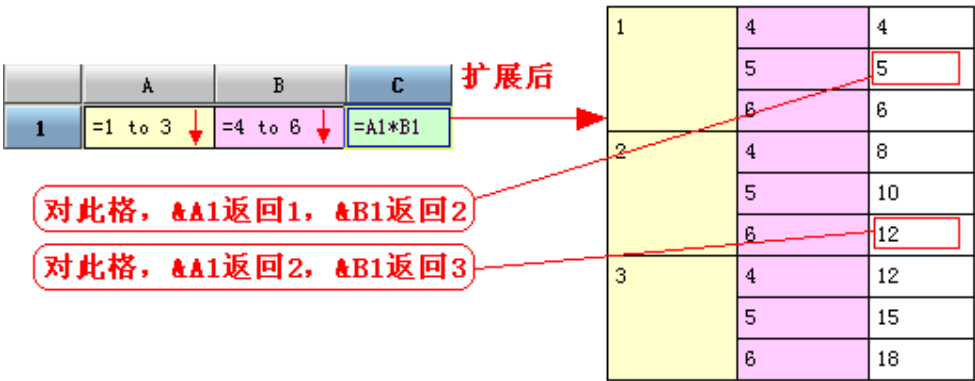
$$\&\text{cellx}$$

返回值：

整数，当前格所属的cellx主格在所有扩展出来的格子中的排序

说明：

cellx必须是当前格的主格，这样&cellx才能够正确运算，其返回值相当于是当前格所属的cellx主格的位置



格集

在实际应用中，往往需要对一组单元格进行运算，而不是单个单元格。这些单元格有可能是固定格，也有可能是主动扩展、被动复制格，为了能在表达式中描述确定的一组单元格，我们引入格集的概念。

格集

格集可以看作满足某种条件的一组单元格的集合。特别的，单个单元格也可以视为仅含一个单元格的格集。

为了说明格集的概念及其作用，我们看看下面的表格：

例1：

扩展前：

A1	B1	C1	D1
A2	B2	C2	D2

扩展后：

a1	b1	c1	d1
a2	b2	c2	d2
		c3	d3
	b4	c4	d4
		c5	d5

a6	b6	c6	d6
		c7	d7
		c8	d8
	b9	c9	d9
		c10	d10
	b11	c11	d11
		c12	d12

填入数据：

客户	产品	日期	金额
Tom	牛奶	2005-5-1	12.00
		2005-5-12	20.00
	毯子	2005-2-21	2.00
		2005-3-1	1.00
Jerry	牛奶	2005-1-1	12.00
		2005-1-12	100.00
		2005-5-1	24.00
	饼干	2005-2-1	3.00
		2005-5-13	4.00
	奶酪	2005-2-1	6.00
		2005-4-1	2.00

如果我们要进行如下统计：①Tom买牛奶的日期；②Tom买毯子的金额；③Jerry在2005-2-1买的商品；④所有买牛奶的金额。

在做这些统计时，涉及到的数据都是发生在一系列格子中的，我们分别来看一下：①c2,c3；②d4,d5；③d9,d11；④d2,d3,d6,d7,d8。

格集表示法

固定格的格集表示法

对于固定的单元格，我们可以用list()函数，：(link) 操作符来表示，书写规则如下：

list(Cell1, Cell2, Cell3,.....Celln) 表示由Cell1, Cell2, Cell3,.....Celln组成的格子的集合

[Cellx : Celly] 其中Cellx与Celly均为单元格，该表达式表示以Cellx与Celly为对角点圈起的矩形区域，而且，Cellx在左上角，Celly在右下角。

提示：Link操作符返回的结果是一个格集，可以对其应用集合函数，如count()、sum()、max()、min()等，但是包含link操作符的单元格不允许设为扩展格。

举例：

list(A1,B3,C4) 表示由A1,B3,C4三个单元格组成的集合

sum([A1:B3]) 表示对以A1与B3为对角点圈起的矩形区域里的格子求和。

扩展格的格集表示法

对于扩展格的格集表示，我们一般和层次坐标结合起来，由层次坐标来界定一个范围，在这个范围内的所有单元格的集合，其书写规则如下：

cellx[层次坐标或者位移坐标]{}

说明：从上述书写规则可以看出，格集相当于在层次坐标或者位移坐标的基础上增加了{}，即可表示该层次坐标或者位移坐标界定的范围内的所有单元格。前文已经提到，如果没有{}，而层次坐标界定的范围内的单元格不止一个，那么该层次坐标返回的是该范围内的第一个单元格，有了{}，就返回该范围内单元格的集合。

例1：

扩展前：

A1	B1	C1	D1
A2	B2	C2	D2

扩展后：

a1	b1	c1	d1
a2	b2	c2	d2
		c3	d3
	b4	c4	d4
		c5	d5
		c6	d6
a7	b7	c7	d7
		c8	d8
		c9	d9
	b10	c10	d10
		c11	d11
		c12	d12

根据上述表格，我们写几个格集并分析其结果由哪些单元格组成。

C2[0]{} 返回扩展后的c2,c3,c4,c5,c6,c7,c8,c9,c10,c11,c12

C2[A2:2]{} 返回扩展后的c7,c8,c9,c10,c11,c12

D2[A2:1,B2:2]{} 返回扩展后的d4,d5,d6

缺省格集

同样的，为了简化格集的表示，很多时候会有缺省的写法，其缺省的规则和层次坐标、位移坐标完全一样，主要也是为了简化其中的层次坐标、位移坐标的写法，即目标单元格和当前格的主格相同时，层次坐标（位移坐标）中的该主格可以省略，如果所有主格都相同，则层次坐标（位移坐标）可以为空，甚至连中括号都可以省略。

其书写规则为：

cellx[缺省的层次坐标或者位移坐标]}

如果层次坐标（位移坐标）完全省略，则缺省的写法为：

cellx[]或Cellx{}

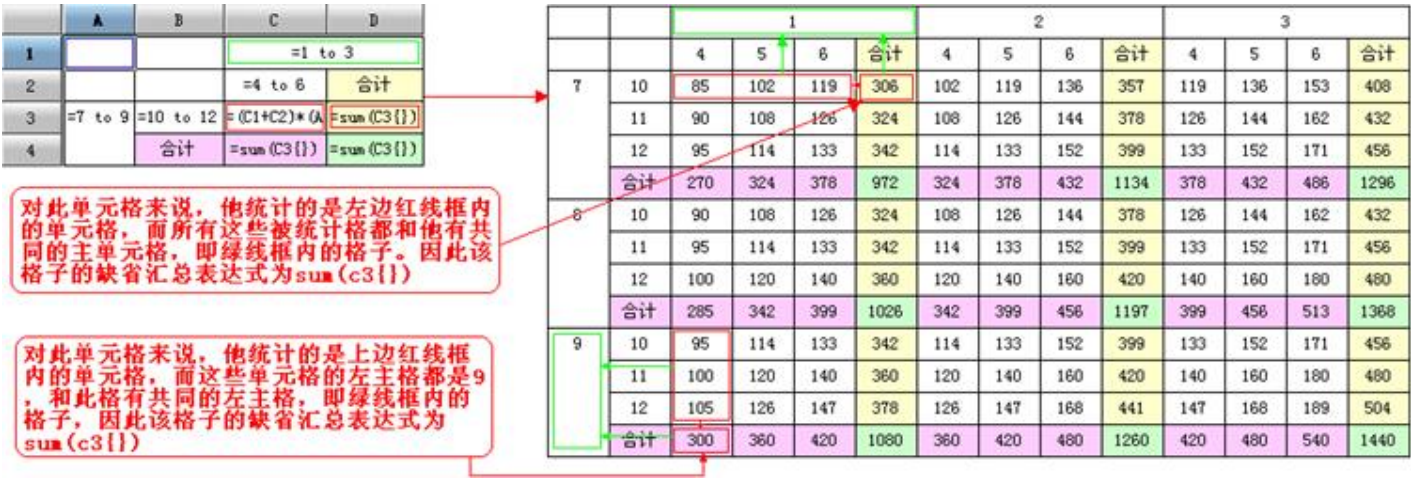
例1：

	A	B	C
1	=1 TO 3	=4 TO 6	=A1*B1
2		合计：	=sum(c1[]{}))

C2中需要统计出当前组中的所有扩展出来的C1单元格，因此所有目标C1的主格都是A1，和当前格C2的主格相同，层次坐标可以采用缺省的写法，C1[]{}即可表示当前组里所有的C1格子

1	4	4
	5	5
	6	6
	合计：	15
2	4	8
	5	10
	6	12
	合计：	30
3	4	12
	5	15
	6	18
	合计：	45

例2：



格集的条件表达式

如前文所述，当需要对一组单元格进行操作时，我们引入了格集表示法，但是格集的表示是和层次坐标、位移坐标紧密相关的，即层次坐标或者位移坐标确定的范围内的单元格的集合，缺省情况下则是当前格所属的主格管辖范围内所有目标单元格的集合。

但是，很多时候，我们需要运算的目标不是层次坐标确定的所有单元格集合，而是该范围内满足某种条件的单元格集合，此时我们引入了条件表达式，其书写规则如下：

```
cellx[层次坐标或者位移坐标]{条件表达式}
```

从上面的表达式可以看出，其含义是对层次坐标或者位移坐标界定的单元格集合再运用条件表达式进行过滤，把符合条件的单元格找出来，并返回。

例1:



\$运算符

请看下面的表格：

年份	月份	销售额	比去年同期
2005年	1月	23	
	3月	43	
	4月	456	
	7月	765	
2006年	4月	876	
	1月	98	
	7月	56	

该表格在设计界面中是如下样子：

年份	月份	销售额	比去年同期
A2	B2	C2	D2

可以看出，设计器中只有一行的表达式单元格，但是扩展后变成了很多的行。而该表格中的难点是计算比去年同期，也就是说，对于2006年7月份的格子来说，需要和2005年7月份的数据进行对比运算，而2006年4月份的格子需要和2005年4月份的格子进行对比运算。

而这个表格中的月份不是连续的，而且不是按顺序排列，因此仅仅靠层次坐标、位移坐标无法定位到去年同月份的单元格，需要借助条件表达式。

此时我们往d2单元格中写入表达式：C2-C2[A2:-1]{当前格的b2主格值==目标格的b2主格值}

这时我们会发现，条件表达式没法写了，当前格的主格是b2，目标格的主格也是b2，如果我们写成b2=b2，显然搞不清谁是谁的，于是我们引入了\$运算符，他在格集条件表达式中指代当前格的主格

例如上面的条件表达式我们可以写成：C2-C2[A2:-1]{\$B2==B2}，其中\$B2指代当前格的B2主格，B2指代目标格的B2主格

总结：\$运算符的书写规则如下：

\$cellx

其含义是在格集表达式中指代当前格的cellx主格

定位和拉伸

报表区域

例1：

		左表头	常规列			
		A(头)	B	C	D	E
分页页眉	1(眉)		润乾软件技术有限公司			
头标题	2(标)		客户购买产品交叉统计表			
	3(标)	报表编号：	runqian001		报表日期：	=now()
	4(标)					
头	5(头)	产品 销量 客户	=ds1.group(			
数据区	6	=ds1.group(CU	=ds1.sum(A			
尾标题	7(注)	操作员：			审核员：	
分页页脚	8(脚)	总页数：			当前页：	

- 1、 横向分区从左到右的顺序为：左表头→常规列→右表头
- 2、 纵向分区从上到下的顺序为：分页页眉→头标题→报表头→分组表头→数据区→表尾区→尾标题→分页页脚。
- 3、 区域顺序从左到右、从上到下不可颠倒
- 4、 除了数据区，其它区域可以不存在
- 5、 合并格不能跨越区域，否则分页时会出现混乱
- 6、 除报表头、分组表头、数据区和表尾区以外的区域适用定位和拉伸规则

空白单元格优先占用

报表设计的时候，由于数据区单元格是横向扩展的，因此经常出现数据区只需要两三列的单元格，而标题区由于不是扩展的，往往需要报表编号、日期、制表人等等信息，导致标题区需要四五列甚至更多的单元格，而报表的展现模型又是严格的矩形，由此导致数据区会出现一些没用的单元格，为了避免这些没用的单元格占用内存，可以把这些单元格设为空白单元格。

所谓空白单元格，就是不存储任何属性的单元格，他仅仅起个占位的作用，保证报表始终是个完整的矩形，因此在内存中他基本不占用内存。

例1:

A(头)	B	C	D	E
	润乾软件技术有限公司			
	客户购买产品交叉统计表			
报表编号：	runqian001		报表日期：	=now()
客户	产品			
购买数量	=dsl.group(dsl.produc			
=dsl.group(dsl.cus	=dsl.sum(quantity)			
操作员：			审核员：	
总页数：			当前页：	

这部分单元格在设
计时没有占用，内
空，
为了
避免
占用
空白
单元
格

规则：

当数据区向右、下进行扩展复制时，如果紧邻的右、下方有空白单元格，那么空白单元格的位置会被优先占用，之后才会继续向右、下复制。

分页时伸缩规则

数据区横向扩展时，很可能横向扩展出很大面积，但是头标题区和尾标题区在设计的时候无法知道数据区会扩展出多大，而为了让报表整体美观，报表的标题区和尾标题区都是希望随着数据区的横向扩展，头标题区和尾标题区有些内容要靠右对齐，有些内容要靠左对齐。这时，我们建议除了希望靠左靠右对齐的单元格外，对中间某一单元格设置“分页时伸缩”属性，这样，横向扩展时报表引擎会对该单元格进行拉大。这就是分页时伸缩规则。

报表头、分组表头、数据区、表尾区横向扩展时，其它区域的单元格从右往左数，找到第一个设置“分页时伸缩”属性的单元格，对其进行拉伸。如果没有单元格设定了分页时伸缩，则对最右边一个单元格进行拉伸。

例1：

A(头)	B	C	D	E
	润乾软件技术有限公司			
	客户购买产品交叉统计表			
报表编号：	runqian001		报表日期：	=now()
客户	产品			
购买数量	=dsl.group(dsl.produc			
=dsl.group(dsl.cus	=dsl.sum(quantity)			
操作员：			审核员：	
总页数：			当前页：	

当数据区横向扩展时，这两部分单元格希望始终在报表的最右边，这样报表整体效果比较方正美观

当数据区横向扩展时，这两部分单元格希望始终在报表最左边，这样报表整体效果最美观

扩展后的效果如下：

客户购买产品交叉统计表						
报表编号：runqian001				报表日期：2006-11-02		
客户	产品 购买数量	大豆	色拉油	苯乙醇	苯甲酸钠	苯磺酸
无锡食品加工厂		300	223	1900	123	2000
天津长城化工厂		0	0	2000	3005	3
江苏天龙食品集团		10	10	0	0	0
中国化工原料厂		0	0	0	0	7
Crystal		2054	27	0	200	0
Chemist		0	0	3000	0	6
Murli		0	0	6	0	9
操作员：李四				审核员：张三		

位置不变

报表头、报表尾、分组头、数据区单元格进行横向扩展时，其它区域的单元格有可能要靠左对齐，因此除了分页时伸缩规则，我们还定义如下规则：

报表头、报表尾、分组表头、数据区单元格进行横向扩展时，其它区域的单元格除了向右靠齐和被拉大的单元格，其余单元格保持位置不变。